

Sampling & Reconstruction of Signals

Complete Exam Preparation Guide

Concepts, Formulas, Python Solutions

Contents

1	Core Concepts	2
1.1	What is Sampling?	2
1.2	Nyquist Theorem	2
1.3	Aliasing	2
1.4	Quantization	2
1.5	FFT / DFT	3
2	Reconstruction — Detailed Theory	4
2.1	Method 1: Zero-Order Hold (ZOH)	4
2.2	Method 2: First-Order Hold (FOH) — Linear Interpolation	4
2.3	Method 3: Polynomial / Spline Interpolation	4
2.4	Method 4: Sinc Interpolation (Ideal Reconstruction)	5
2.5	Why Sinc Works — Frequency Domain Explanation	5
2.6	Comparison Table	6
3	Problems and Solutions	7
3.1	Problem 1: Basic Sampling and Visualization	7
3.2	Problem 2: Demonstrate Aliasing	7
3.3	Problem 3: FFT / Frequency Spectrum	8
3.4	Problem 4: Quantization and SQNR	8
3.5	Problem 5: Reconstruction — Compare All Four Methods	10
3.6	Problem 6: Sinc Reconstruction — Step by Step	11
3.7	Problem 7: Effect of Sampling Rate on Reconstruction	11
3.8	Problem 8: Reconstruction with Missing Samples	12
3.9	Problem 9: Downsampling and Upsampling	12
3.10	Problem 10: Anti-Aliasing Filter + Sampling	13
3.11	Problem 11: Complete ADC/DAC Pipeline	14
3.12	Problem 12: Frequency-Domain Reconstruction View	15
4	Plotting Reference (Matplotlib)	16
4.1	Core Plot Types	16
4.2	Labels, Title, Legend, Grid	16
4.3	Subplots	16
4.4	Common Format Strings	17
4.5	Logarithmic Scale	17
5	Quick Reference Cheat Sheet	18
5.1	Key Formulas	18
5.2	Essential NumPy / SciPy Functions	18
5.3	Reconstruction Quick Code	18

1 Core Concepts

1.1 What is Sampling?

Sampling

Sampling is the process of converting a **continuous-time signal** $x(t)$ into a **discrete-time signal** $x[n]$ by taking values at regular intervals T_s .

$$x[n] = x(nT_s), \quad n = 0, 1, 2, \dots$$

where $T_s = \frac{1}{f_s}$ is the **sampling period** and f_s is the **sampling frequency** (samples per second).

1.2 Nyquist Theorem

Nyquist–Shannon Sampling Theorem

A bandlimited signal with maximum frequency $f_{\max}$ can be **perfectly reconstructed** from its samples if and only if:

$$f_s \geq 2 f_{\max}$$

- **Nyquist Rate** = $2 f_{\max}$: the minimum sampling rate required.
- **Nyquist Frequency** = $f_s/2$: the maximum frequency that can be represented at sampling rate f_s .

1.3 Aliasing

Aliasing

When $f_s < 2 f_{\max}$, frequencies above $f_s/2$ **fold back** into the range $[0, f_s/2]$ and appear as lower frequencies. This is **aliasing** — it corrupts the signal irreversibly.

Aliased frequency formula:

$$f_{\text{alias}} = \left| f_{\text{signal}} - \text{round}\left(\frac{f_{\text{signal}}}{f_s}\right) \cdot f_s \right|$$

Simple folding rule:

$$f_{\text{apparent}} = f \bmod f_s, \quad \text{if } f_{\text{apparent}} > \frac{f_s}{2}, \quad f_{\text{apparent}} = f_s - f_{\text{apparent}}$$

1.4 Quantization

Quantization

After sampling (time discretisation), we also discretise the **amplitude** using B bits.

- Number of levels: $L = 2^B$
- Step size: $\Delta = \frac{x_{\max} - x_{\min}}{2^B}$

- **SQNR** (for sinusoidal input):

$$\text{SQNR} \approx 6.02 B + 1.76 \text{ dB}$$

Each additional bit adds ≈ 6 dB of quality.

- Actual SQNR:

$$\text{SQNR} = 10 \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right) = 10 \log_{10} \left(\frac{\text{mean}(x^2)}{\text{mean}(e^2)} \right)$$

1.5 FFT / DFT

Frequency Spectrum via FFT

The FFT decomposes a time-domain signal into its frequency components.

- `np.fft.fft(x)` returns N complex values.
- `np.fft.fftfreq(N, d=1/fs)` returns the corresponding frequency axis.
- Magnitude spectrum: $|X[k]|/N$ (normalised). Multiply by 2 for single-sided spectrum.
- The spectrum is symmetric for real signals; plot only $[0, f_s/2]$.

2 Reconstruction — Detailed Theory

What is Reconstruction?

Reconstruction is the process of recovering a **continuous-time signal** from its **discrete samples**. It answers: *“How do we fill in the gaps between samples?”*

Perfect reconstruction is possible *only* when the Nyquist condition is satisfied.

2.1 Method 1: Zero-Order Hold (ZOH)

Zero-Order Hold

Hold each sample value **constant** until the next sample:

$$x_{\text{ZOH}}(t) = x[n] \quad \text{for } nT_s \leq t < (n+1)T_s$$

Properties:

- Produces a **staircase** waveform.
- Simplest method; used in DACs.
- Worst accuracy — flat line cannot follow curves.
- Frequency response has a sinc roll-off distortion.

2.2 Method 2: First-Order Hold (FOH) — Linear Interpolation

First-Order Hold (Linear Interpolation)

Draw **straight lines** between consecutive samples:

$$x_{\text{FOH}}(t) = x[n] + \frac{x[n+1] - x[n]}{T_s} (t - nT_s), \quad nT_s \leq t < (n+1)T_s$$

Properties:

- Continuous output (no jumps).
- Has **corners** (kinks) at sample points — not smooth.
- Much better than ZOH for most signals.

2.3 Method 3: Polynomial / Spline Interpolation

Spline Interpolation

Fit **smooth polynomial curves** through sample points.

- **Cubic Spline:** Fits a cubic polynomial $ax^3 + bx^2 + cx + d$ between each pair of samples, ensuring:
 1. Curve passes through every sample point.
 2. First derivative is continuous (smooth).
 3. Second derivative is continuous (very smooth).
- Very smooth, no corners.

- Practical and computationally efficient.
- **Not** theoretically perfect, but very close for oversampled signals.

2.4 Method 4: Sinc Interpolation (Ideal Reconstruction)

Whittaker–Shannon Sinc Interpolation

The ideal reconstruction formula:

$$x(t) = \sum_{n=-\infty}^{\infty} x[n] \operatorname{sinc}\left(\frac{t - nT_s}{T_s}\right)$$

where

$$\operatorname{sinc}(u) = \frac{\sin(\pi u)}{\pi u}$$

Key properties of sinc:

- $\operatorname{sinc}(0) = 1$ (full contribution at its own sample).
- $\operatorname{sinc}(\pm 1) = \operatorname{sinc}(\pm 2) = \dots = 0$ (zero at all other sample points).
- Each sample generates a shifted sinc; summing all sincs recovers the signal.

Properties:

- **Perfect reconstruction** if $f_s \geq 2f_{\max}$.
- Equivalent to ideal low-pass filtering in the frequency domain.
- Requires infinite samples (sinc extends to $\pm\infty$) — not practical in finite signals.
- Computationally expensive.

2.5 Why Sinc Works — Frequency Domain Explanation

Reconstruction

1. **Sampling in time** = multiplying by an impulse train.
2. **Multiplication in time** = convolution in frequency.
3. Convolution with impulse train = **periodic copies** of the spectrum, repeated every f_s Hz.
4. If $f_s > 2f_{\max}$, the copies **don't overlap**.
5. An **ideal low-pass filter** with cutoff $f_s/2$ keeps only the original copy and removes all others.
6. The **sinc function** is the time-domain impulse response of the ideal LPF.

Sampling $\xrightarrow{\text{freq. domain}}$ Spectral copies $\xrightarrow{\text{LPF at } f_s/2}$ Original spectrum recovered

2.6 Comparison Table

Method	Smoothness	Accuracy	Complexity	Use Case
Zero-Order Hold	Worst	Lowest	Simplest	DACs
First-Order Hold	Good	Medium	Simple	Real-time
Cubic Spline	Very Good	High	Medium	Graphics, audio
Sinc (Ideal)	Perfect	Perfect*	Highest	Theory

*Perfect only when Nyquist condition is satisfied.

3 Problems and Solutions

3.1 Problem 1: Basic Sampling and Visualization

Problem 1

Generate $x(t) = \sin(2\pi \cdot 5t) + \sin(2\pi \cdot 12t)$. Sample it at $f_s = 50$ Hz for 1 second. Verify the Nyquist condition.

Concept: $f_{\max} = 12$ Hz, Nyquist rate = 24 Hz. Since $50 > 24$, no aliasing.

```

1 import numpy as np
2
3 f1, f2 = 5, 12
4 fs = 50
5 Ts = 1 / fs
6
7 # "Continuous" signal (very fine time step)
8 t_cont = np.arange(0, 1, 0.0001)
9 x_cont = np.sin(2*np.pi*f1*t_cont) + np.sin(2*np.pi*f2*t_cont)
10
11 # Sampled signal
12 t_s = np.arange(0, 1, Ts)
13 x_s = np.sin(2*np.pi*f1*t_s) + np.sin(2*np.pi*f2*t_s)

```

3.2 Problem 2: Demonstrate Aliasing

Problem 2

Generate $x(t) = \sin(2\pi \cdot 10t)$. Sample at $f_s = 25$ Hz (safe) and $f_s = 15$ Hz (aliased). Compute the aliased frequency.

Concept: Nyquist rate = 20 Hz. At $f_s = 15$: $f_{\text{alias}} = |10 - \text{round}(10/15) \times 15| = |10 - 15| = 5$ Hz.

```

1 import numpy as np
2
3 f_signal = 10
4 t_cont = np.arange(0, 1, 0.0001)
5 x_cont = np.sin(2*np.pi*f_signal*t_cont)
6
7 for fs in [25, 15]:
8     Ts = 1 / fs
9     t_s = np.arange(0, 1, Ts)
10    x_s = np.sin(2*np.pi*f_signal*t_s)
11
12    # Aliased frequency
13    f_alias = abs(f_signal - round(f_signal/fs)*fs)
14    print(f"fs={fs}: f_alias = {f_alias} Hz")

```

3.3 Problem 3: FFT / Frequency Spectrum

Problem 3

Generate $x(t) = 3\sin(2\pi \cdot 20t) + 1.5\sin(2\pi \cdot 50t)$. Sample at $f_s = 200$ Hz for 1 second. Compute and display the frequency components using FFT.

```

1 import numpy as np
2
3 fs = 200
4 N = 200                                # 1 second * 200 Hz
5 t = np.arange(0, 1, 1/fs)
6 x = 3*np.sin(2*np.pi*20*t) + 1.5*np.sin(2*np.pi*50*t)
7
8 X = np.fft.fft(x)
9 freq = np.fft.fftfreq(N, d=1/fs)
10 mag = np.abs(X) / N * 2                # single-sided amplitude
11
12 # Keep only positive frequencies
13 pos = freq >= 0
14 freq_pos = freq[pos]
15 mag_pos = mag[pos]
16
17 # Print detected peaks
18 threshold = 0.1
19 peaks = np.where(mag_pos > threshold)[0]
20 for i in peaks:
21     print(f"Frequency: {freq_pos[i]:.1f} Hz, Amplitude: {mag_pos[i]:.2f}")

```

Expected output: Peaks at 20 Hz (amplitude ≈ 3) and 50 Hz (amplitude ≈ 1.5).

3.4 Problem 4: Quantization and SQNR

Problem 4

Sample a sine wave at $f_s = 1000$ Hz. Quantize using $B = 4$ and $B = 8$ bits. Calculate SQNR and compare with the theoretical value.

```

1 import numpy as np
2
3 fs = 1000
4 t = np.arange(0, 0.1, 1/fs)
5 x = np.sin(2*np.pi*100*t)            # 100 Hz sine wave, range [-1, 1]
6
7 for B in [4, 8]:
8     L = 2**B
9     delta = 2.0 / L                    # (1 - (-1)) / L
10
11     # Quantize
12     x_idx = np.floor((x - (-1)) / delta).astype(int)
13     x_idx = np.clip(x_idx, 0, L-1)
14     x_q = -1 + x_idx * delta + delta/2
15
16     # SQNR
17     error = x - x_q
18     SQNR_actual = 10*np.log10(np.mean(x**2) / np.mean(error**2))
19     SQNR_theory = 6.02*B + 1.76
20

```

```
21     print(f"B={B}: SQNR_actual={SQNR_actual:.2f} dB, "  
22           f"SQNR_theory={SQNR_theory:.2f} dB")
```

3.5 Problem 5: Reconstruction — Compare All Four Methods

Problem 5 (KEY PROBLEM)

Generate $x(t) = \sin(2\pi \cdot 3t) + 0.5\sin(2\pi \cdot 7t)$. Sample at $f_s = 30$ Hz. Reconstruct using ZOH, FOH, Cubic Spline, and Sinc interpolation. Compute the RMS error for each.

```

1 import numpy as np
2 from scipy.interpolate import CubicSpline
3
4 f1, f2 = 3, 7
5 fs = 30
6 Ts = 1/fs
7
8 # Reference continuous signal
9 t_cont = np.arange(0, 1, 0.0001)
10 x_cont = np.sin(2*np.pi*f1*t_cont) + 0.5*np.sin(2*np.pi*f2*t_cont)
11
12 # Sample
13 t_s = np.arange(0, 1, Ts)
14 x_s = np.sin(2*np.pi*f1*t_s) + 0.5*np.sin(2*np.pi*f2*t_s)
15
16 # ----- METHOD 1: Zero-Order Hold -----
17 def zoh(t_r, t_samp, x_samp):
18     x_r = np.zeros(len(t_r))
19     for i in range(len(t_r)):
20         idx = np.searchsorted(t_samp, t_r[i], side='right') - 1
21         idx = max(0, idx)
22         x_r[i] = x_samp[idx]
23     return x_r
24
25 x_zoh = zoh(t_cont, t_s, x_s)
26
27 # ----- METHOD 2: First-Order Hold (Linear) -----
28 x_foh = np.interp(t_cont, t_s, x_s)
29
30 # ----- METHOD 3: Cubic Spline -----
31 cs = CubicSpline(t_s, x_s)
32 x_cubic = cs(t_cont)
33
34 # ----- METHOD 4: Sinc Interpolation -----
35 def sinc_recon(t_r, t_samp, x_samp, Ts):
36     x_r = np.zeros(len(t_r))
37     for n in range(len(t_samp)):
38         x_r += x_samp[n] * np.sinc((t_r - t_samp[n]) / Ts)
39     return x_r
40
41 x_sinc = sinc_recon(t_cont, t_s, x_s, Ts)
42
43 # ----- RMS Errors -----
44 def rms(a, b):
45     return np.sqrt(np.mean((a - b)**2))
46
47 print(f"ZOH:      RMS = {rms(x_cont, x_zoh):.6f}")
48 print(f"FOH:      RMS = {rms(x_cont, x_foh):.6f}")
49 print(f"Cubic:     RMS = {rms(x_cont, x_cubic):.6f}")
50 print(f"Sinc:      RMS = {rms(x_cont, x_sinc):.6f}")

```

Expected ranking (best to worst): Sinc > Cubic Spline > FOH > ZOH.

3.6 Problem 6: Sinc Reconstruction — Step by Step

Problem 6

Sample $x(t) = \sin(2\pi \cdot 2t)$ at $f_s = 10$ Hz. Compute the individual sinc contribution from each sample and show how they sum to form the reconstructed signal.

```

1 import numpy as np
2
3 f = 2
4 fs = 10
5 Ts = 1/fs
6
7 t_cont = np.arange(0, 1, 0.0001)
8 x_cont = np.sin(2*np.pi*f*t_cont)
9
10 t_s = np.arange(0, 1, Ts)      # 10 samples
11 x_s = np.sin(2*np.pi*f*t_s)
12 N = len(t_s)
13
14 # Individual sinc contributions
15 sinc_contributions = np.zeros((N, len(t_cont)))
16 for n in range(N):
17     sinc_contributions[n] = x_s[n] * np.sinc((t_cont - t_s[n]) / Ts)
18
19 # Progressive reconstruction: sum of first k sines
20 for k in [1, 3, 5, N]:
21     partial = np.sum(sinc_contributions[:k], axis=0)
22     err = np.sqrt(np.mean((x_cont - partial)**2))
23     print(f"Using {k:2d} sines: RMS error = {err:.6f}")

```

Key insight: As more sines are added, the error decreases. With all N sines, reconstruction is nearly perfect.

3.7 Problem 7: Effect of Sampling Rate on Reconstruction

Problem 7

Reconstruct $x(t) = \sin(2\pi \cdot 10t)$ using sinc interpolation at $f_s = 15, 20, 25, 40, 100$ Hz. Show how quality varies with sampling rate.

```

1 import numpy as np
2
3 f = 10
4 nyquist_rate = 2*f      # 20 Hz
5
6 t_cont = np.arange(0, 1, 0.0001)
7 x_cont = np.sin(2*np.pi*f*t_cont)
8
9 for fs in [15, 20, 25, 40, 100]:
10     Ts = 1/fs
11     t_s = np.arange(0, 1, Ts)
12     x_s = np.sin(2*np.pi*f*t_s)
13
14     # Sinc reconstruction
15     x_r = np.zeros(len(t_cont))
16     for n in range(len(t_s)):
17         x_r += x_s[n] * np.sinc((t_cont - t_s[n]) / Ts)

```

```

18
19     rms = np.sqrt(np.mean((x_cont - x_r)**2))
20     status = "ALIASED" if fs < nyquist_rate else "OK"
21     print(f"fs={fs:3d} Hz: RMS={rms:.6f} [{status}]")

```

Key insight: Error drops dramatically once f_s crosses the Nyquist rate, and keeps improving with oversampling.

3.8 Problem 8: Reconstruction with Missing Samples

Problem 8

Sample $x(t) = \sin(2\pi \cdot 5t)$ at $f_s = 50$ Hz. Randomly remove 30% of the samples. Reconstruct using linear interpolation and cubic spline. Compare.

```

1 import numpy as np
2 from scipy.interpolate import CubicSpline
3
4 np.random.seed(42)
5
6 f, fs = 5, 50
7 t_cont = np.arange(0, 1, 0.0001)
8 x_cont = np.sin(2*np.pi*f*t_cont)
9
10 t_all = np.arange(0, 1, 1/fs)
11 x_all = np.sin(2*np.pi*f*t_all)
12 N = len(t_all)
13
14 # Keep 70% of samples randomly
15 keep = np.sort(np.random.choice(N, int(0.7*N), replace=False))
16 if 0 not in keep: keep = np.concatenate(([0], keep))
17 if N-1 not in keep: keep = np.concatenate((keep, [N-1]))
18
19 t_avail = t_all[keep]
20 x_avail = x_all[keep]
21
22 # Linear reconstruction
23 x_linear = np.interp(t_cont, t_avail, x_avail)
24
25 # Cubic spline reconstruction
26 cs = CubicSpline(t_avail, x_avail)
27 x_cubic = cs(t_cont)
28
29 rms_l = np.sqrt(np.mean((x_cont - x_linear)**2))
30 rms_c = np.sqrt(np.mean((x_cont - x_cubic)**2))
31
32 print(f"Samples kept: {len(keep)} / {N}")
33 print(f"Linear RMS: {rms_l:.6f}")
34 print(f"Cubic RMS: {rms_c:.6f}")

```

3.9 Problem 9: Downsampling and Upsampling

Problem 9

Given a signal sampled at $f_s = 100$ Hz, perform downsampling by $M = 4$ and upsampling by $L = 3$. Show the result.

```

1 import numpy as np
2
3 fs = 100
4 t = np.arange(0, 0.5, 1/fs)
5 x = np.sin(2*np.pi*10*t) + 0.5*np.sin(2*np.pi*20*t)
6
7 # --- Downsampling by M=4 ---
8 M = 4
9 x_down = x[::M]          # keep every M-th sample
10 t_down = t[::M]
11 fs_down = fs / M         # new fs = 25 Hz
12 print(f"Downsample: {len(x)} -> {len(x_down)} samples, fs={fs_down} Hz")
13
14 # --- Upsampling by L=3 (zero insertion) ---
15 L = 3
16 x_up = np.zeros(len(x) * L)
17 x_up[::L] = x             # place samples every L positions
18 fs_up = fs * L           # new fs = 300 Hz
19 print(f"Upsample: {len(x)} -> {len(x_up)} samples, fs={fs_up} Hz")
20
21 # After upsampling, a low-pass filter is needed to interpolate
22 # (otherwise there are spectral images)

```

Downsampling Warning

New Nyquist frequency = $f_s/(2M)$. Any signal components above this will alias! Always apply a low-pass (anti-aliasing) filter before downsampling.

3.10 Problem 10: Anti-Aliasing Filter + Sampling

Problem 10

A signal has components at 30, 60, 100, 150, 180 Hz. You must sample at $f_s = 250$ Hz ($f_{\text{Nyquist}} = 125$ Hz). Apply an anti-aliasing filter before sampling.

```

1 import numpy as np
2 from scipy import signal as sig
3
4 fs_high = 5000             # simulate continuous
5 t = np.arange(0, 0.5, 1/fs_high)
6 freqs = [30, 60, 100, 150, 180]
7
8 x = sum(np.sin(2*np.pi*f*t) for f in freqs)
9
10 # Design anti-aliasing filter
11 fs_target = 250
12 f_nyquist = fs_target / 2   # 125 Hz
13 Wn = f_nyquist / (fs_high / 2) # normalised cutoff
14 b, a = sig.butter(8, Wn, btype='low')
15
16 # Apply filter (removes 150 Hz and 180 Hz)
17 x_filtered = sig.filtfilt(b, a, x)
18
19 # Sample
20 factor = fs_high // fs_target
21 x_sampled = x_filtered[::factor]
22

```

```

23 kept      = [f for f in freqs if f < f_nyquist]
24 removed   = [f for f in freqs if f >= f_nyquist]
25 print(f"Kept:      {kept}")
26 print(f"Removed:  {removed}")

```

3.11 Problem 11: Complete ADC/DAC Pipeline

Problem 11

Implement the full pipeline: Original $\rightarrow$ Anti-alias filter $\rightarrow$ Sample $\rightarrow$ Quantize $\rightarrow$ Reconstruct.
Compute the total reconstruction error.

```

1 import numpy as np
2 from scipy import signal as sig
3 from scipy.interpolate import CubicSpline
4
5 fs_cont    = 50000
6 fs_target  = 10000
7 f_nyquist  = fs_target / 2      # 5000 Hz
8 T = 0.01
9
10 t = np.arange(0, T, 1/fs_cont)
11 freqs = [100, 300, 800, 1500, 3000, 5500]
12
13 x = sum(np.sin(2*np.pi*f*t) / (1+f/440) for f in freqs)
14
15 # Stage 1: Anti-aliasing filter
16 Wn = f_nyquist / (fs_cont/2)
17 b, a = sig.butter(6, Wn, 'low')
18 x_filt = sig.filtfilt(b, a, x)
19
20 # Stage 2: Sampling
21 M = fs_cont // fs_target
22 t_s = t[::M]
23 x_s = x_filt[::M]
24
25 # Stage 3: Quantization (8 bits)
26 B = 8
27 L = 2**B
28 xmin, xmax = x_s.min()-0.1, x_s.max()+0.1
29 delta = (xmax - xmin) / L
30 x_idx = np.clip(np.floor((x_s - xmin)/delta).astype(int), 0, L-1)
31 x_q    = xmin + x_idx*delta + delta/2
32
33 SQNR = 10*np.log10(np.mean(x_s**2) / np.mean((x_s - x_q)**2))
34
35 # Stage 4: Reconstruction (cubic spline)
36 cs = CubicSpline(t_s, x_q)
37 x_recon = cs(t)
38
39 # Total error
40 rms = np.sqrt(np.mean((x - x_recon)**2))
41 print(f"SQNR = {SQNR:.2f} dB")
42 print(f"Total RMS error = {rms:.6f}")

```

3.12 Problem 12: Frequency-Domain Reconstruction View

Problem 12

Show that sampling creates spectral copies, and that an ideal low-pass filter at $f_s/2$ recovers the original spectrum. Sample $x(t) = \sin(2\pi \cdot 5t)$ at $f_s = 40$ Hz.

```

1 import numpy as np
2
3 f_signal = 5
4 fs = 40
5
6 # Spectral copies appear at f_signal + k*fs for all integers k
7 print("Spectral copies of the signal after sampling:")
8 for k in range(-2, 3):
9     f_plus = k*fs + f_signal
10    f_minus = k*fs - f_signal
11    in_band = "<-- KEEP" if abs(f_plus)<=fs/2 or abs(f_minus)<=fs/2 else ""
12    print(f"    k={k:+d}: {f_plus:+.0f} Hz, {f_minus:+.0f} Hz    {in_band}")
13
14 print(f"\nIdeal LPF cutoff = fs/2 = {fs/2} Hz")
15 print(f"Only +/-{f_signal} Hz survive -> original recovered!")

```

Output:

```

k=-2: -75 Hz, -85 Hz
k=-1: -35 Hz, -45 Hz
k= 0:  +5 Hz,  -5 Hz  <-- KEEP
k=+1: +45 Hz, +35 Hz
k=+2: +85 Hz, +75 Hz

```

4 Plotting Reference (Matplotlib)

All plotting uses:

```
1 import matplotlib.pyplot as plt
```

4.1 Core Plot Types

Task	Code
Continuous line	<code>plt.plot(t, x, 'b-', linewidth=2, label='Signal')</code>
Discrete stems (samples)	<code>plt.stem(t_s, x_s, linefmt='r-', markerfmt='ro', basefmt='k-')</code>
Step / staircase (ZOH)	<code>plt.step(t, x, 'r-', where='post')</code>
Vertical reference line	<code>plt.axvline(x=val, color='r', linestyle='--')</code>
Horizontal reference line	<code>plt.axhline(y=val, color='k', linestyle='--')</code>
Fill between curves	<code>plt.fill_between(t, x1, x2, alpha=0.3)</code>
Bar chart	<code>plt.bar(x_pos, heights, color='b')</code>

4.2 Labels, Title, Legend, Grid

```
1 plt.xlabel('Time (s)')
2 plt.ylabel('Amplitude')
3 plt.title('My Plot Title')
4 plt.legend()                # show legend
5 plt.grid(True)              # show grid
6 plt.xlim(0, 1)              # x-axis limits
7 plt.ylim(-2, 2)             # y-axis limits
8 plt.tight_layout()          # prevent label overlap
9 plt.show()
```

4.3 Subplots

```
1 # Method 1: Simple
2 fig, axes = plt.subplots(2, 1, figsize=(12, 8))
3 axes[0].plot(t, x1)
4 axes[0].set_title('First')
5 axes[1].plot(t, x2)
6 axes[1].set_title('Second')
7 plt.tight_layout()
8 plt.show()
9
10 # Method 2: Grid
11 fig, axes = plt.subplots(2, 3, figsize=(15, 8)) # 2 rows, 3 cols
12 axes[0][1].plot(t, x) # row 0, col 1
```

4.4 Common Format Strings

Color	Line Style	Marker
'b' blue	'-' solid	'o' circle
'r' red	'--' dashed	's' square
'g' green	'-.' dash-dot	'^' triangle
'k' black	':' dotted	'x' cross
'm' magenta		'.' point

Combined: 'ro--' = red circles with dashed line.

4.5 Logarithmic Scale

```
1 plt.yscale('log')      # log scale on y-axis (useful for error plots)
2 plt.xscale('log')
```

5 Quick Reference Cheat Sheet

5.1 Key Formulas

Quantity	Formula
Sampling period	$T_s = 1/f_s$
Nyquist rate	$f_{s,\min} = 2 f_{\max}$
Nyquist frequency	$f_{\text{Nyquist}} = f_s/2$
Aliased frequency	$f_{\text{alias}} = f - \text{round}(f/f_s) \cdot f_s $
Quantization step	$\Delta = (x_{\max} - x_{\min})/2^B$
SQNR (sinusoidal)	$6.02 B + 1.76 \text{ dB}$
Sinc reconstruction	$x(t) = \sum_n x[n] \text{sinc}\left(\frac{t - nT_s}{T_s}\right)$
RMS error	$\sqrt{\text{mean}((x_{\text{orig}} - x_{\text{recon}})^2)}$

5.2 Essential NumPy / SciPy Functions

Function	Purpose
<code>np.arange(start, stop, step)</code>	Evenly spaced array
<code>np.sin(2*np.pi*f*t)</code>	Sine wave
<code>np.sinc(x)</code>	$\sin(\pi x)/(\pi x)$
<code>np.fft.fft(x)</code>	Compute FFT
<code>np.fft.fftfreq(N, d=1/fs)</code>	Frequency axis for FFT
<code>np.abs(X)</code>	Magnitude of complex array
<code>np.interp(t_new, t, x)</code>	Linear interpolation
<code>np.searchsorted(a, v)</code>	Binary search (for ZOH)
<code>np.clip(x, lo, hi)</code>	Clamp values to range
<code>np.floor(x)</code>	Round down
<code>x[:,M]</code>	Take every M -th element
<code>CubicSpline(t_s, x_s)</code>	Cubic spline interpolator
<code>interp1d(t, x, kind=...)</code>	General interpolator
<code>sig.butter(order, Wn)</code>	Butterworth filter design
<code>sig.filtfilt(b, a, x)</code>	Zero-phase filtering

5.3 Reconstruction Quick Code

```
1 # Zero-Order Hold
```

```
2 idx = np.searchsorted(t_s, t_new, side='right') - 1
3 x_zoh = x_s[np.clip(idx, 0, len(x_s)-1)]
4
5 # First-Order Hold (Linear)
6 x_foh = np.interp(t_new, t_s, x_s)
7
8 # Cubic Spline
9 from scipy.interpolate import CubicSpline
10 x_cubic = CubicSpline(t_s, x_s)(t_new)
11
12 # Sinc (Ideal)
13 x_sinc = sum(x_s[n]*np.sinc((t_new - t_s[n])/Ts) for n in range(len(t_s)))
14
15 # RMS Error
16 rms = np.sqrt(np.mean((x_original - x_reconstructed)**2))
```

Good Luck on Your Exam!